



Introduction to Software Engineering

Week 1- Welcome

Pankaj Agrawal



Who Am I

- Pankaj Agrawal
- Graduated from University of Toronto, MIS
- 15+ Professional Experience as a Solution Architect , Software Developer , and Advanced Analytics
- Currently working as Head, Data & Analytics @ IMCO , Ex-RBC



What to Expect

- Software Engineering Practise
- 1.5 lecture and 30 min Q&A

Topics:

- Different types of software methodology
- Product Management
- Importance of working in a team
- Practical Technologies
- Distributed / Scalable System

Goal:

- Make the course as aligned to the real world.
- Improve Coding Skills
- Think as professional Software Engineer

** This course is not a web development course and is aimed at teaching and making you a holistic software engineer*



Tutorials

- 10 Tutorial in total out of which 4 will be labs, 10% in total lab grades; 2 tutorial will be dedicated for DevOps & Deployment.
- Four tutorial will be dedicated for sprint demo's of the final project. It will done via Zoom
- Tutorial will start from week of Jan 12



Course Structure

Type	Description	Tentative Due Date	Weight
Tutorial/ Labs	Tutorial/ Labs (4 in total)	NA	10%
Assignments	2 Assignments	Around Feb 15 th , Mar 22 nd	10%
Final Exam	Final Exam	TBD	35%
Project	Ongoing project and will be delivered in phases	On-going, final presentation in last two lectures	45%
Total			100%

There will be a total of 4 labs, each accounting for 2.5%



Final Test

- Term test will contain multiple choice / short answer / coding questions / software development scenarios.
- It will include everything covered until the day of the test.



Term Project

- Develop a software or an app from the projects available
- Work in a agile team of 4-5 people
- Build a MVP of working software
- Delivery will be in sprints (stages) , concluded by a final team presentation. All team members of the team will present their work and contribution in the project
- Part of the evaluation will also be peer evaluation
- Details will be available soon

With 9 weeks to complete the project, implementation must be of fair complexity. (not that of a basic CRUD application, like Web Gallery) It is recommended that progress be made every week towards project completion.

You have unlimited chances to modify your proposal at any point in the course and be reevaluated. You can schedule the instructor's office hours for re-evaluation.



Contact Info

Email: pankaj.agrawal@utoronto.ca

Office hours:

- Friday, 19:00-20:00 hours (By email appointment only)

Piazza Discussion Board

Slack

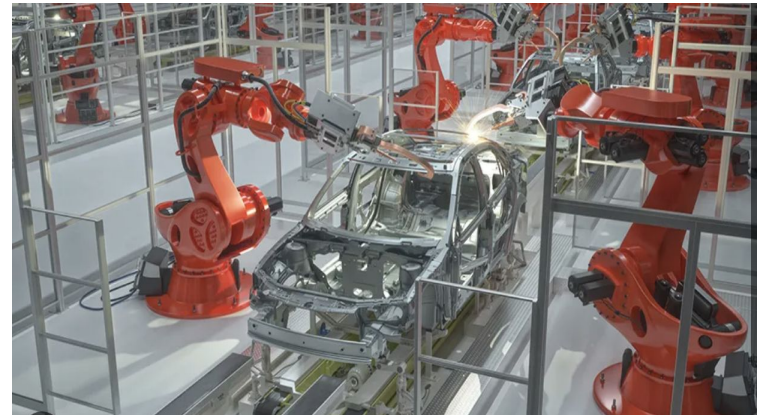


Software Engineering

What is **engineering**? And how is it different from **hacking/programming**?

Producing a car / bridge

- Estimable costs and risks
- Expected results
- High quality
- Separation between plan and production
- Simulation before construction
- Quality assurance through measurement
- Potential for automation





Software Engineering ??

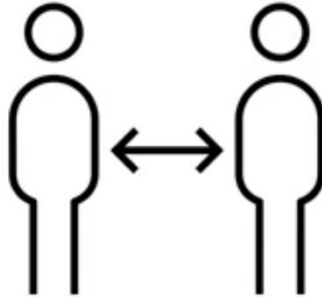
“Software engineering is the application of scientific principles to the design and creation of software”

The field uses a systematic approach to collect and analyze business requirements in order to design, build, and test software applications to satisfy those business requirements.

Software Engineer vs Software Developer

Engineer

- Also a developer
- Broad , big picture knowledge base
- Systematic development process
- Focus on structure
- Designing, building, and maintaining software systems.
- Consulting with stakeholders



Developer

- Narrower in scope than SE
- Creative approach
- Write code to implement specific functionality and solve specific problem

Why Learn Software Engineering

The \$85 Billion Cost of Bad Code

A [recent study](#) from Stripe and Harris Poll found the average developer spends 42% of their time dealing with technical debt and maintenance issues, of which 3.8 hours are spent just on debugging “bad code,” or poor quality code that’s difficult to maintain. The opportunity cost of bad code comes to \$85 billion annually — resources that could otherwise be used to build better software.



Source: pullrequest.com/blog/cost-of-bad-code

Technical Debt - A poor, hard to understand “hacky” implementation which will have to be repaid with interest later on.

- 1 hour saved now, could cost 20+ hours later.

Good Code

```
a.comp == null && b.comp == null
```

Bad Code

```
(a.comp == null && !(b.comp != null)) ||  
(b.comp == null && !(a.comp != null)) ||  
(a.comp == null && b.comp == null)
```

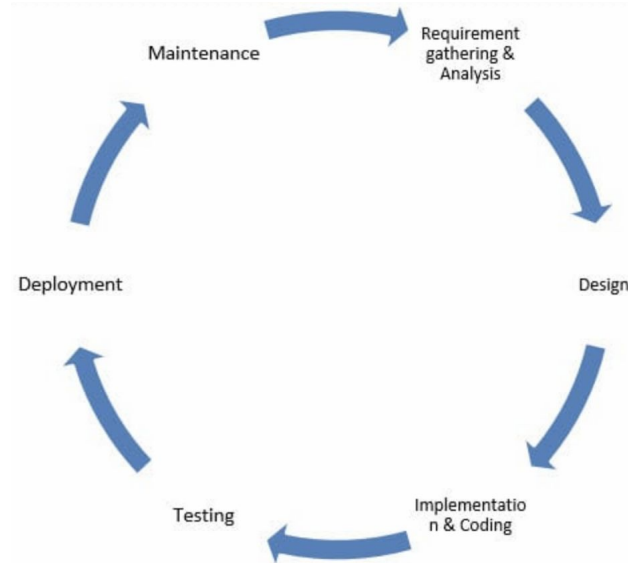
20% of projects fail. Many of these are due to poor development practices.



Introduction to Software Development Lifecycle

- Guides the software development process
- Systematic process to develop high quality software
- Identify discrete steps needed to develop software
- Meets the requirement of the client
- Cycle of planning, design and implement
- Minimizes the development risks and costs

Phases in SDLC

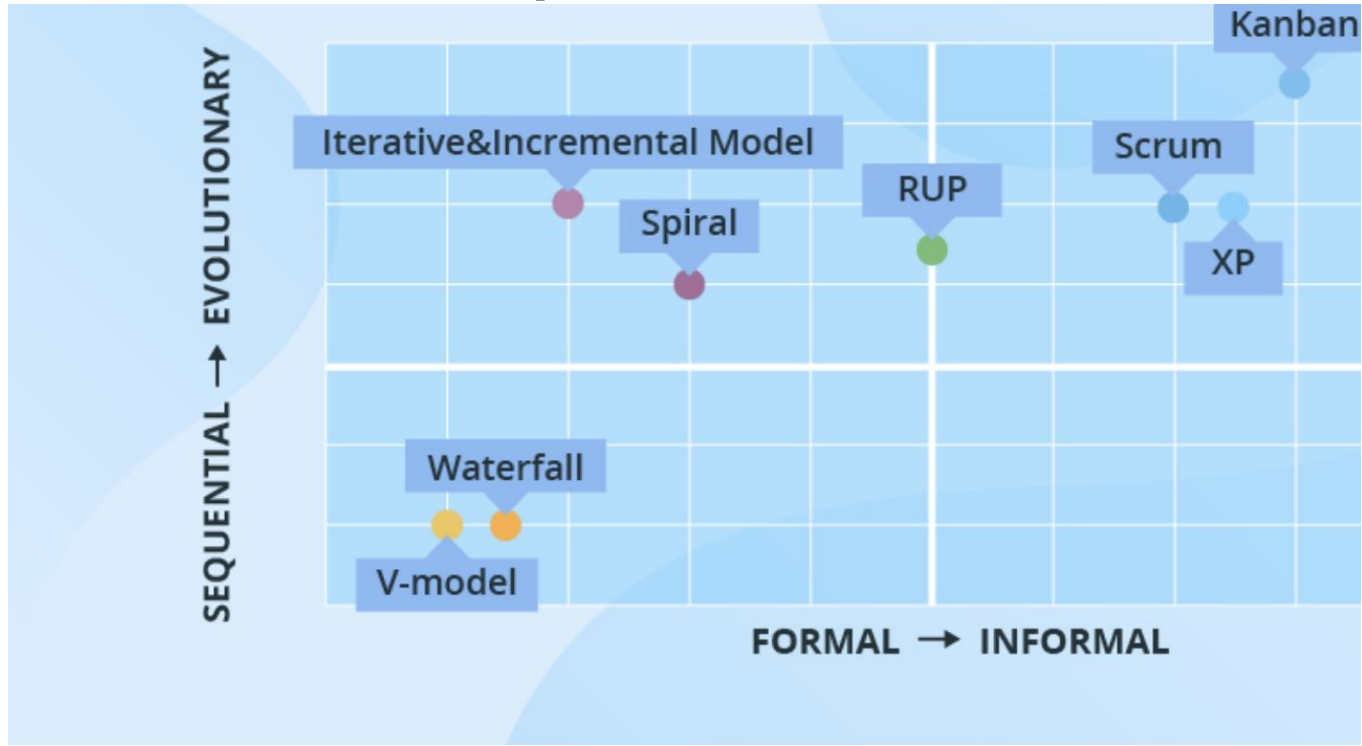


- Each phase is discreet
- Some organization might have different names and may have few or more stages

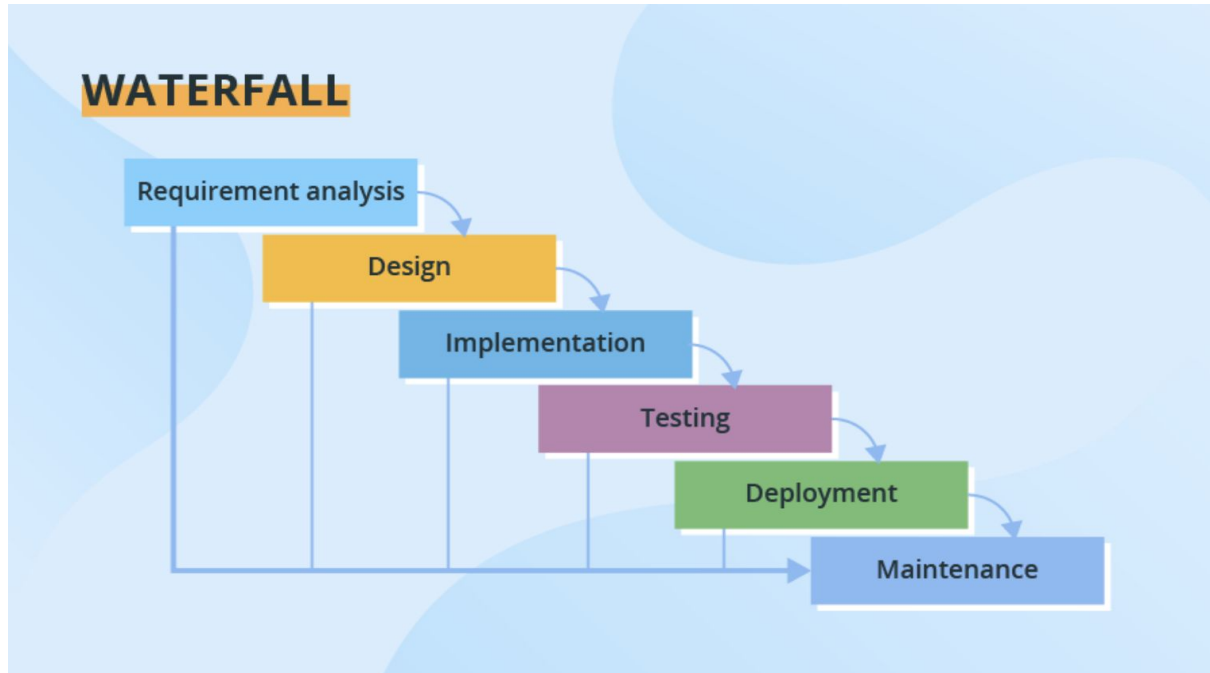
Six Stages of SDLC

- Planning - requirement gathering and development of the SRS.
 - Design - The architecture is developed during the design phase and the design document is created.
 - Development - Coding takes place
 - Testing - Issues with the code are found and fixed if possible.
 - Deployment - Releasing the code to the production environment.
 - Maintenance - Feedback is collected from stakeholders, other UI issues may be identified, and code enhancements suggested.
-
- This information then can be fed into another software development cycle if necessary.

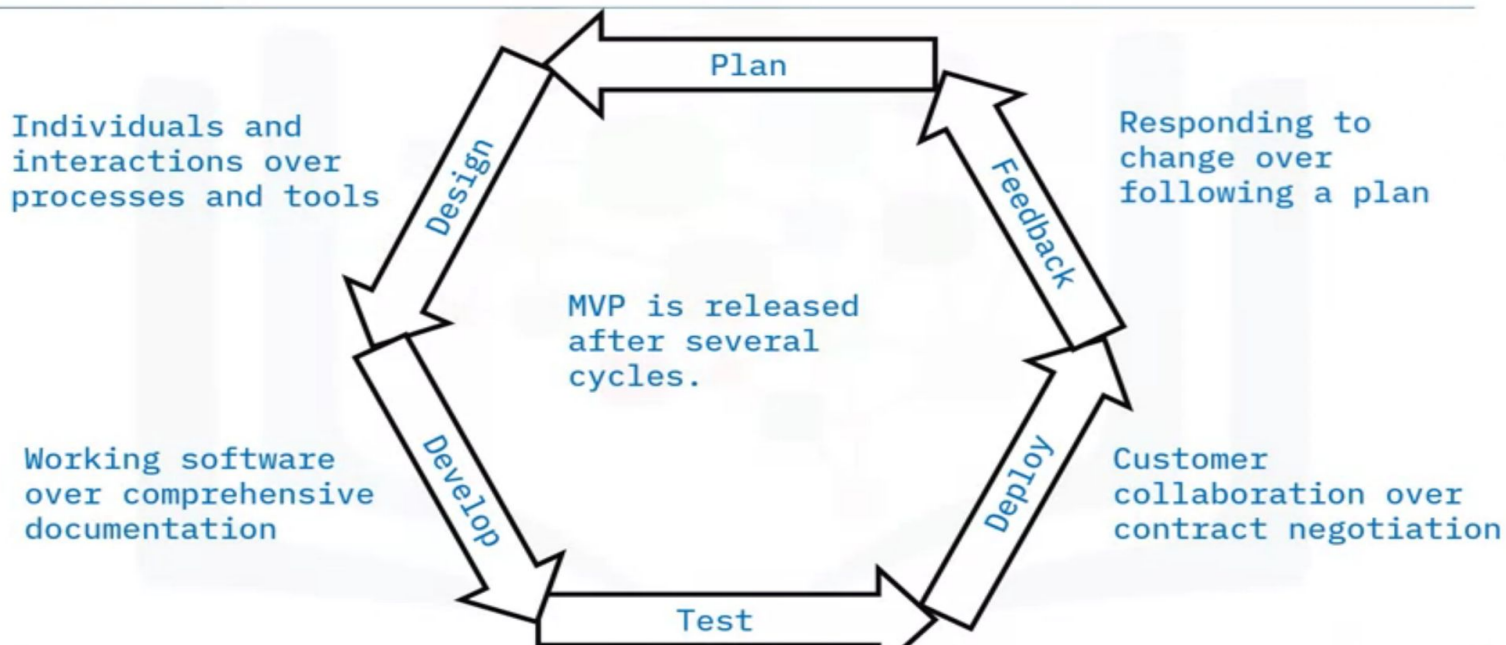
Software Development Model



Waterfall Model



Agile



Use Cases

Projects where failures and downtimes are unacceptable (e.g., medical software, aviation fleet management software).



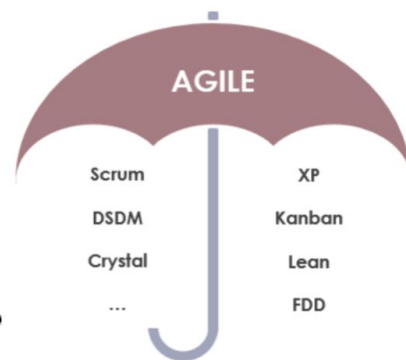
Agile



www.dilbert.com scottadams@aol.com



© 2007 Scott Adams, Inc./Dist. by UFS, Inc.





Agile contd.

A project management approach that seeks to respond to change and unpredictability, primarily using incremental, iterative work sequences (often called “**sprints**”).

- Also: a collection of practices to facilitate that approach.
- All predicated on the principles outlined in “The Manifesto for Agile Software Development.”



Agile Software Development Is....

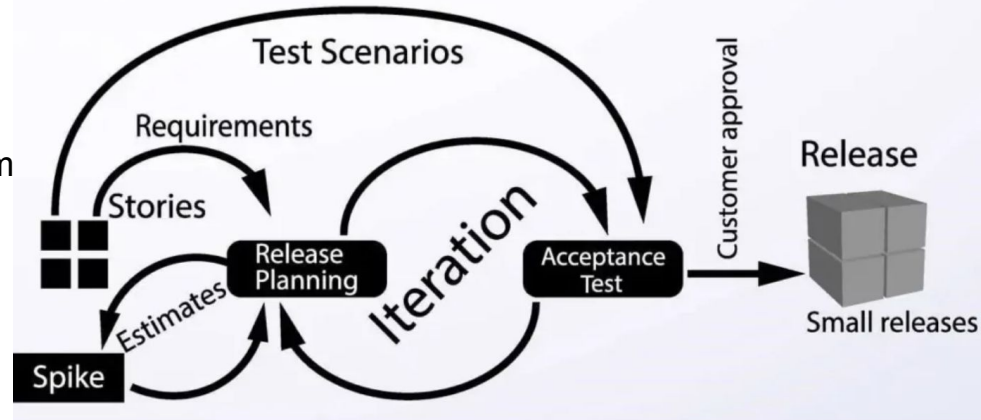
- A set of software engineering best practices (allowing for rapid delivery of high quality software)
- A business approach (alignment of business / client needs with goals)

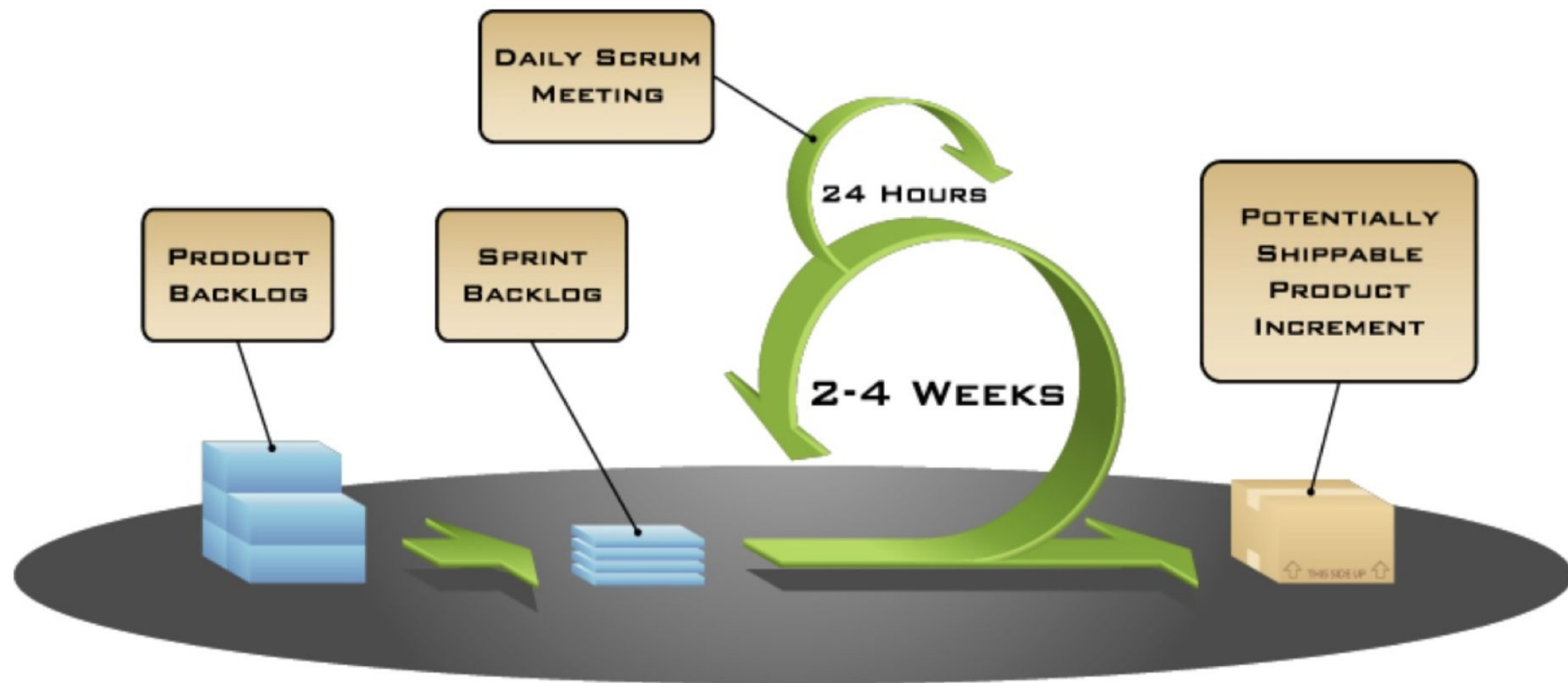
Extreme programming

- Iterative Incremental Model
- Emphasise on technical aspects of software development
- Customer's decision drives the project
- In a nutshell, about good practices taken to an extreme
- for ex.

Since pair-programming is good, let's do it all of the time. Since testing early is good, let's test before the production code is even written

Extreme Programming







Scrum Elements

Product

- Product Backlog
- Sprint backlog

Process

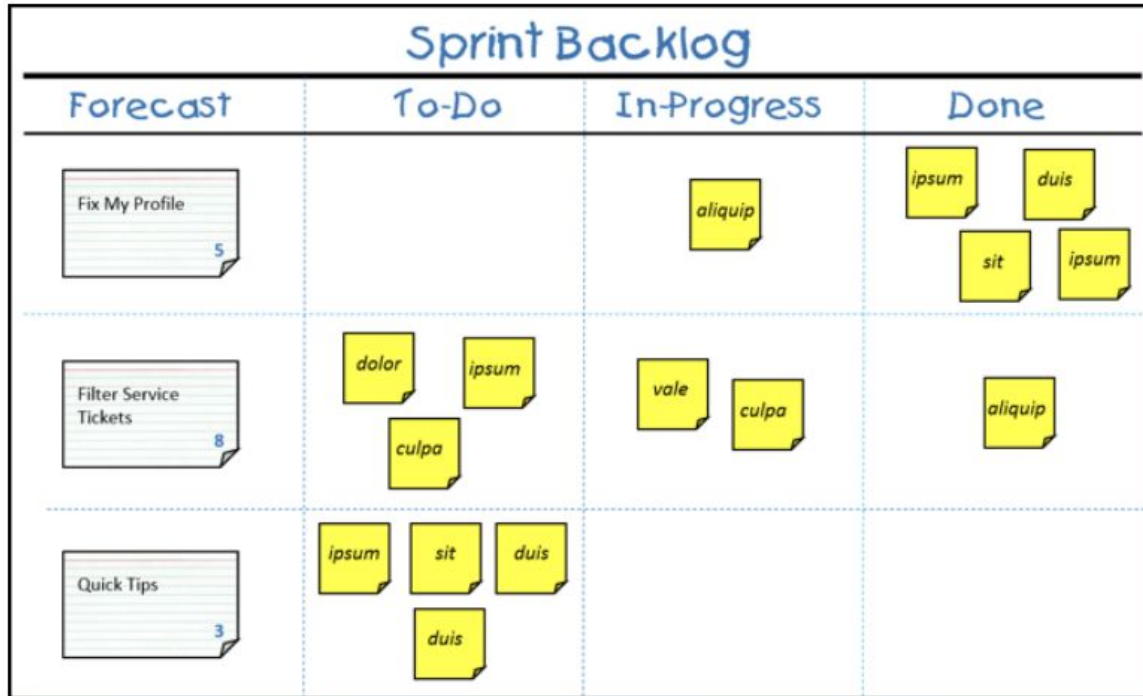
- Sprint planning and backlog meeting
- Daily scrum meeting
- Sprint Retrospective
- Sprint Review Meeting



Product Backlog/ Sprint Backlog

- The product backlog is all the features of the product
- The sprint backlog is all the features that will be worked on for that sprint. It should be broken down into discrete tasks:
 - Refined
 - Estimated
 - Assigned to individual , who will work on the story
 - Acceptance criteria should be clear
- User stories are used in sprints to deliver work

Sprint Backlog



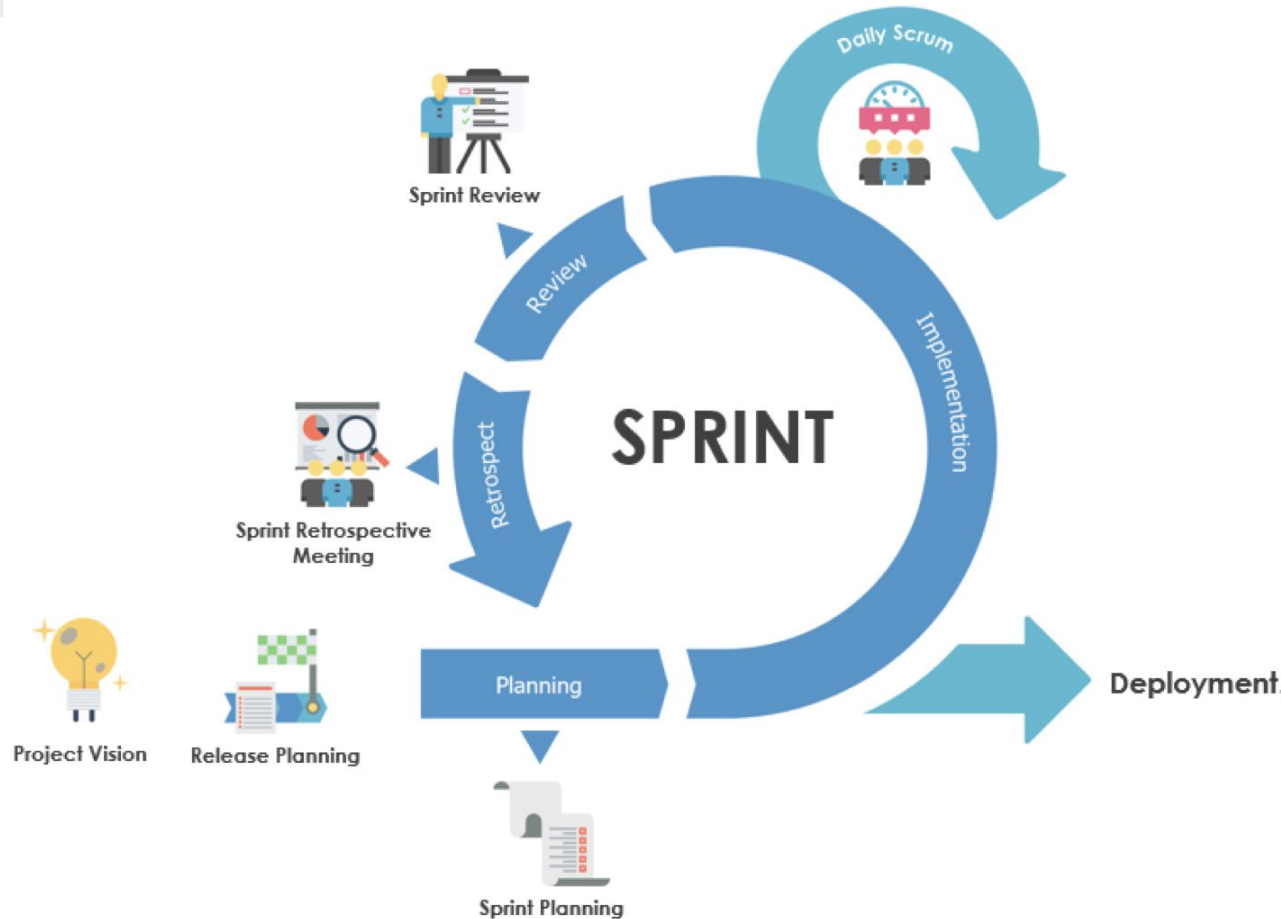


Scrum Meetings

- Sprint Planning Meeting
 - Entire team decide together what to tackle for that sprint
- Daily Scrum Meeting
 - Quick meeting to touch base on:
 - What you did yesterday ? What you plan to do today ? Do you need help
- Sprint Retro - Review the sprint progress
- Sprint Review - Review the product

Scrum Process

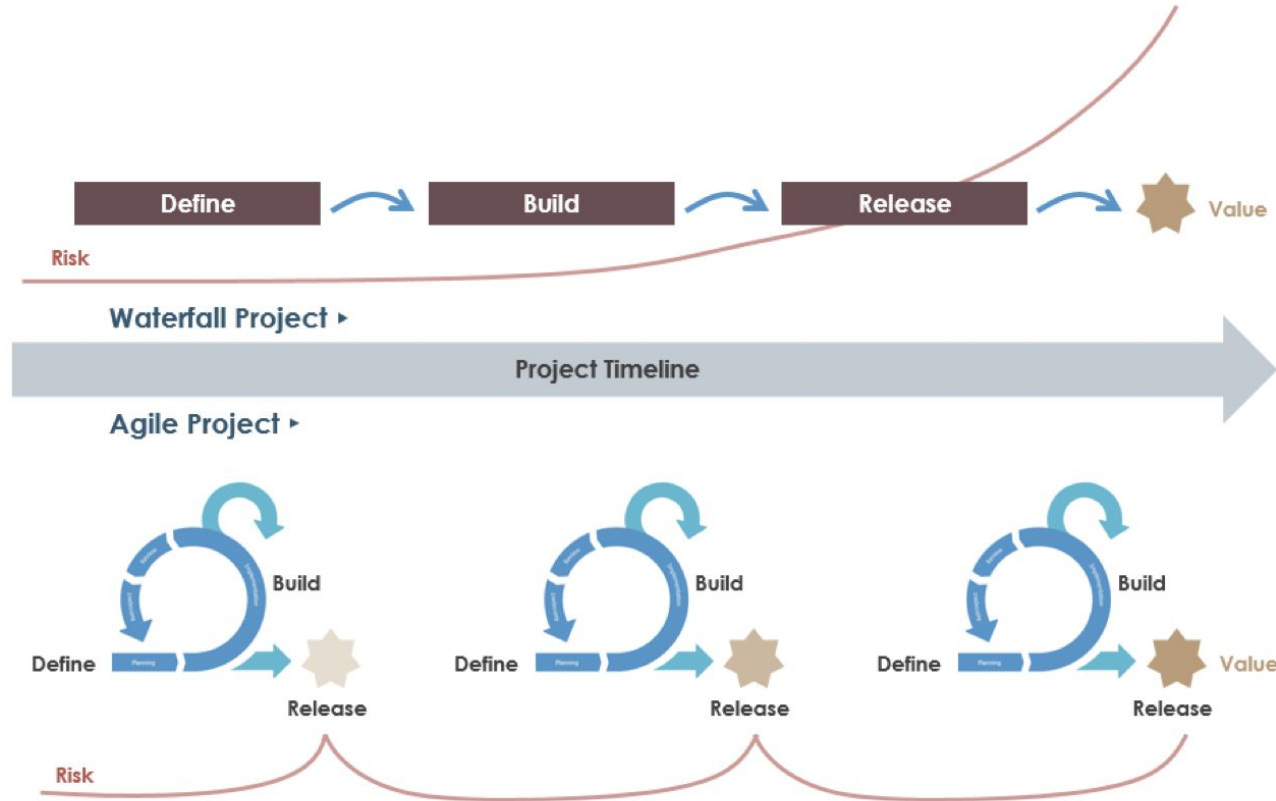
Sprint Cycle



Small Releases

The system is put into production in a few months, before solving the whole problem.

New releases are made often—anywhere from daily to monthly.



Twelve Principles of Agile



Individuals and interactions

1. Projects are built around motivated individuals, who should be trusted
2. Face-to-face conversation is the best form of communication (co-location)
3. Self-organizing teams

Working software

4. Working software is delivered frequently (weeks rather than months)
5. Working software is the principal measure of progress
6. Sustainable development, able to maintain a constant pace
7. Continuous attention to technical excellence and good design
8. Simplicity—the art of maximizing the amount of work not done—is essential

Customer collaboration

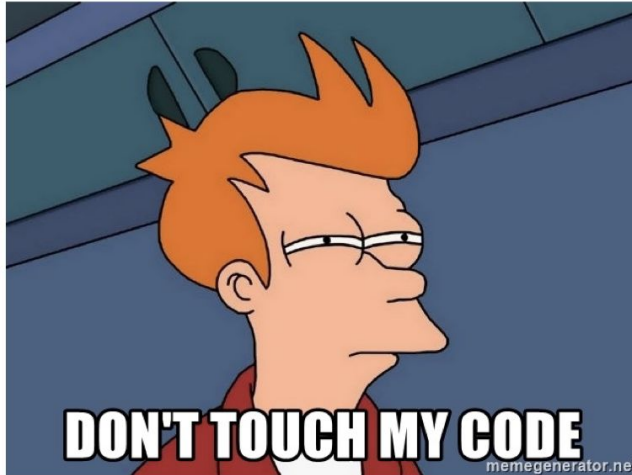
9. Customer satisfaction by rapid delivery of useful software
10. Close, daily cooperation between business people and developers

Responding to change

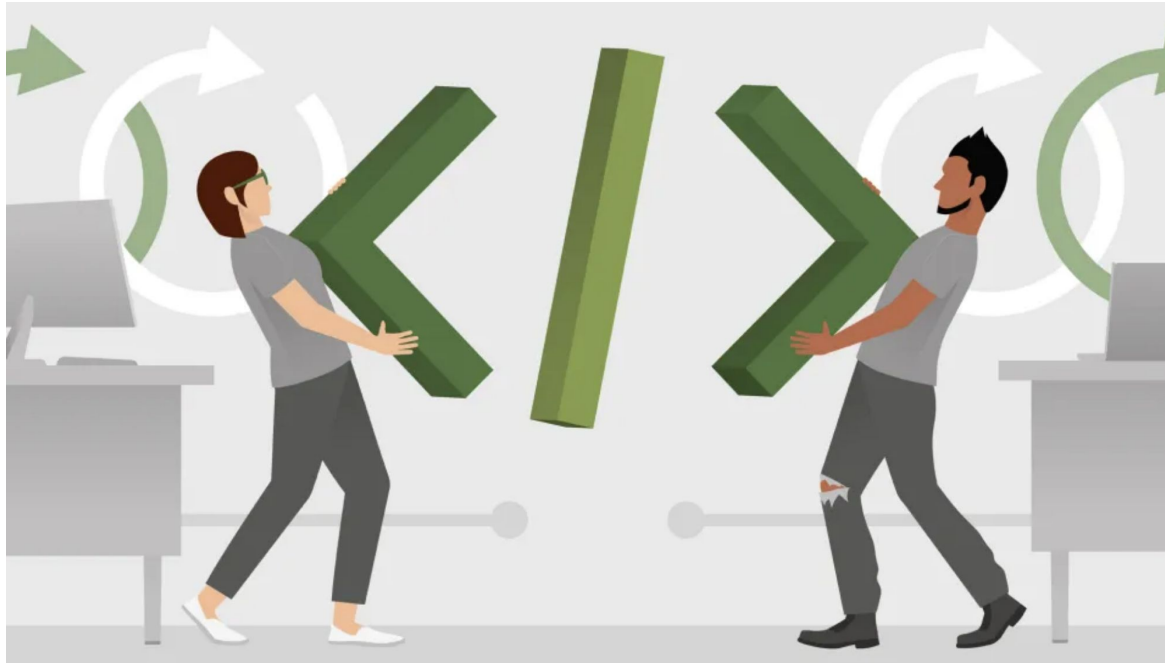
11. Welcome changing requirements, even late in development
12. Regular adaptation to changing circumstances

Collective Ownership

Any developer can change any line of code to add functionality, fix bugs, improve designs or refactor. No one person becomes a bottleneck for changes



Pair programming





Pair Programming Style

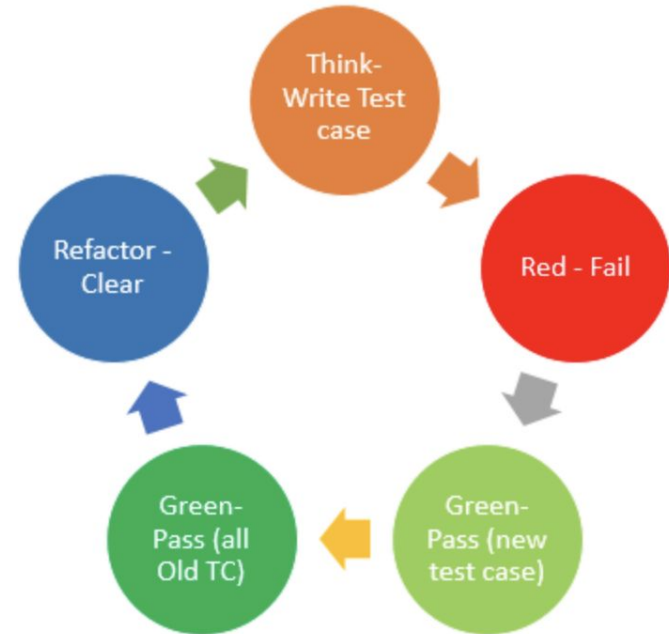
- Driver / Navigator
 - “Driver” types the code
 - “Navigator” reviews the code as it’s written
- Ping - Pong
 - First developer writes a test
 - Second developer writes a code to pass the test
- Strong Style
 - Pair a junior programmer with senior one
 - Senior programmer is navigator
 - Junior programmer is driver



<https://www.youtube.com/watch?v=ET3Q6zNK3Io>

Test Driven Development

- Programmers write a failing unit test case for the simplest piece of functionality that you need to implement.
- Simplest code to pass the test
- Refactor the code to pass or meet the standards of the code



Git



git is a distributed version control system that enables teams of software engineers to collaboratively work on repositories of code.

An *open source project* started by Linus Torvalds and the most popular version control system used by software engineers currently.

Modifications to the codebase are represented as “commit” objects, and the git history is just an immutable, linked list of commits

Many of you are probably exposed to basic commands (add, commit, push, pull, clone), however we would like you to use best practices.

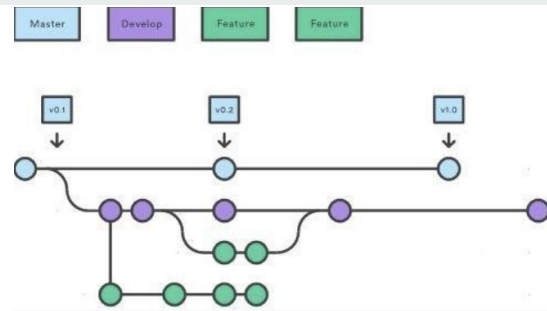
gitflow

git flow is a commonly used git branch structure that promotes agile software development & continuous integration/continuous deployment.

the master branch stores the official release history, and the develop branch is used to integrate features.

- feature branches are merged into develop
- once develop has been thoroughly tested and contains all the features/fixes for a release, you merge develop into master!
- if there are issues with a feature, we can revert the feature branch in develop
- if there are issues with a release, we can revert the merge commit in master

result: Isolating our environments prevents issues from becoming a problem!



typical git usage

what is the typical command flow for contributing to a software project?

1. **branch off develop and create your feature branch**
2. implement your feature on your feature branch
3. **add the untracked files to your next commit**
4. **create your commit**
5. **push the commit**
6. merge feature branch into develop

```
> git checkout -b DEV-101
```

```
> git add --all
```

```
> git commit -m "msg"
```

```
> git push
```


What if someone pushes code to the main branch that I need for my feature branch?

there are **two methods** for bringing code in from other branches into your own.



merging

combines branches together into a commit



Rebase + merging

modify, mutate and move commits on your branch with rebase and **then** merge



Merge

> **git merge <branch>**

Merging creates a commit which combines the tip of the master branch (HEAD) and the tip of the feature branch into one commit

- this commit is referred to as a “merge commit”
- the merge commit becomes the new HEAD after the merge is complete

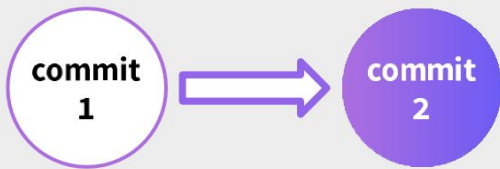
Pros:

- non-destructive,
- doesn't alter any existing branches
- simpler to manage, not as complicated as rebase

Cons:

- tends to pollute the master branch with extra merge commits.
- Becomes an issue in high traffic projects with many contributors

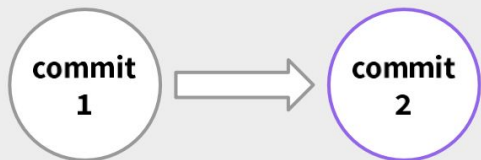
git merge



master

new branch

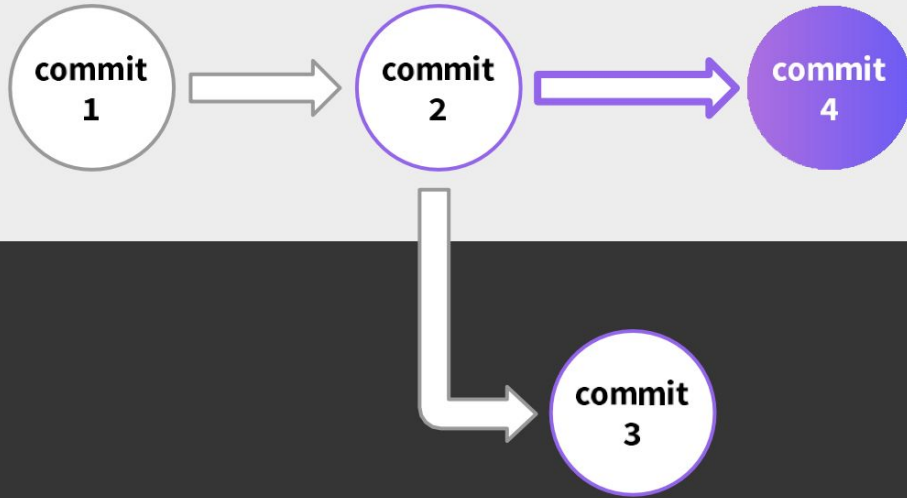
git merge



master

new branch

git merge



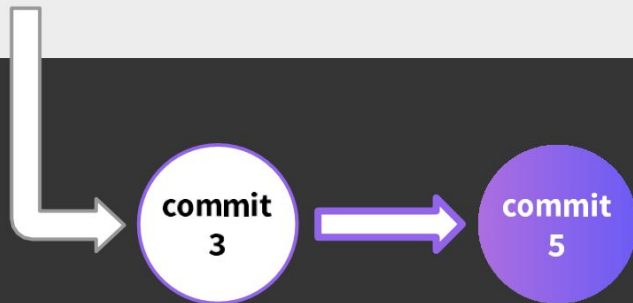
master

new branch

git merge

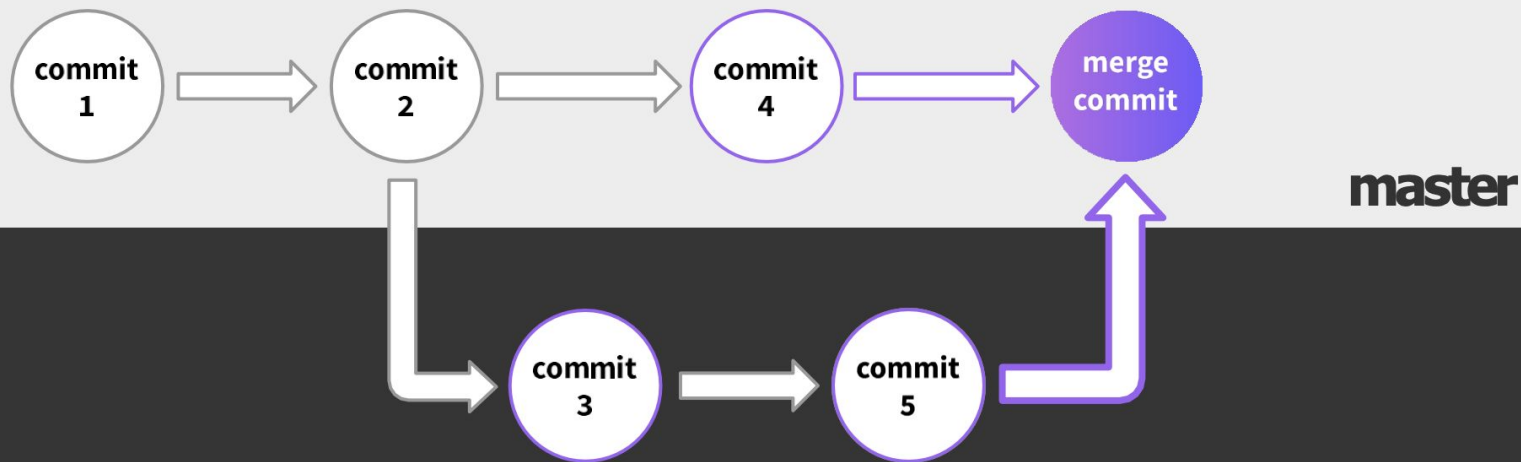


master



new branch

git merge



new branch



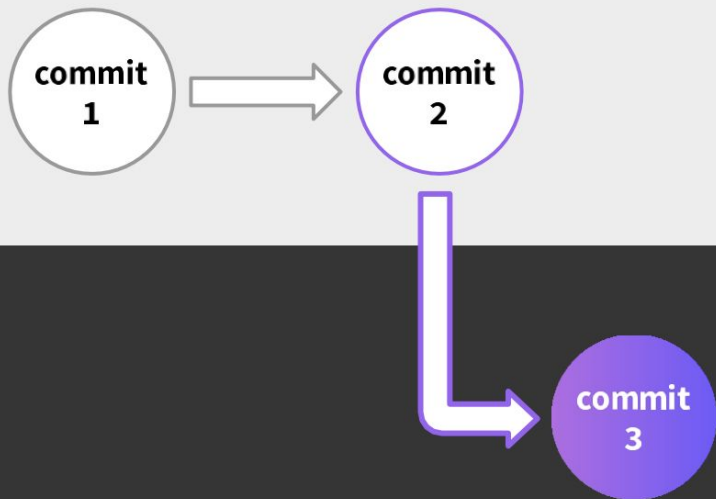
Rebasing + Merge

>git rebase <branch>

rebasing moves things around, the first commit of feature branch is placed sequentially after the tip of the branch you're rebasing onto

- no extra commit merging the two
- partially rewrites the git history by creating brand new commits for each commit in the master branch
- requires extra step to merge: `rebasefeaturebranch` on master, then merge

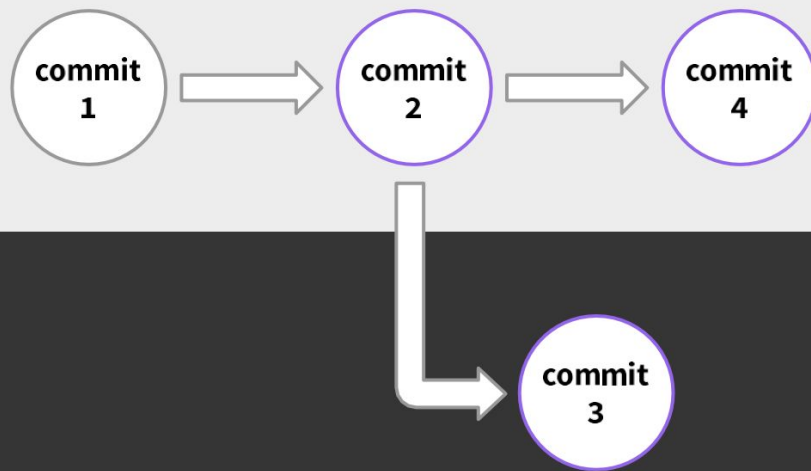
git rebase



master

new branch

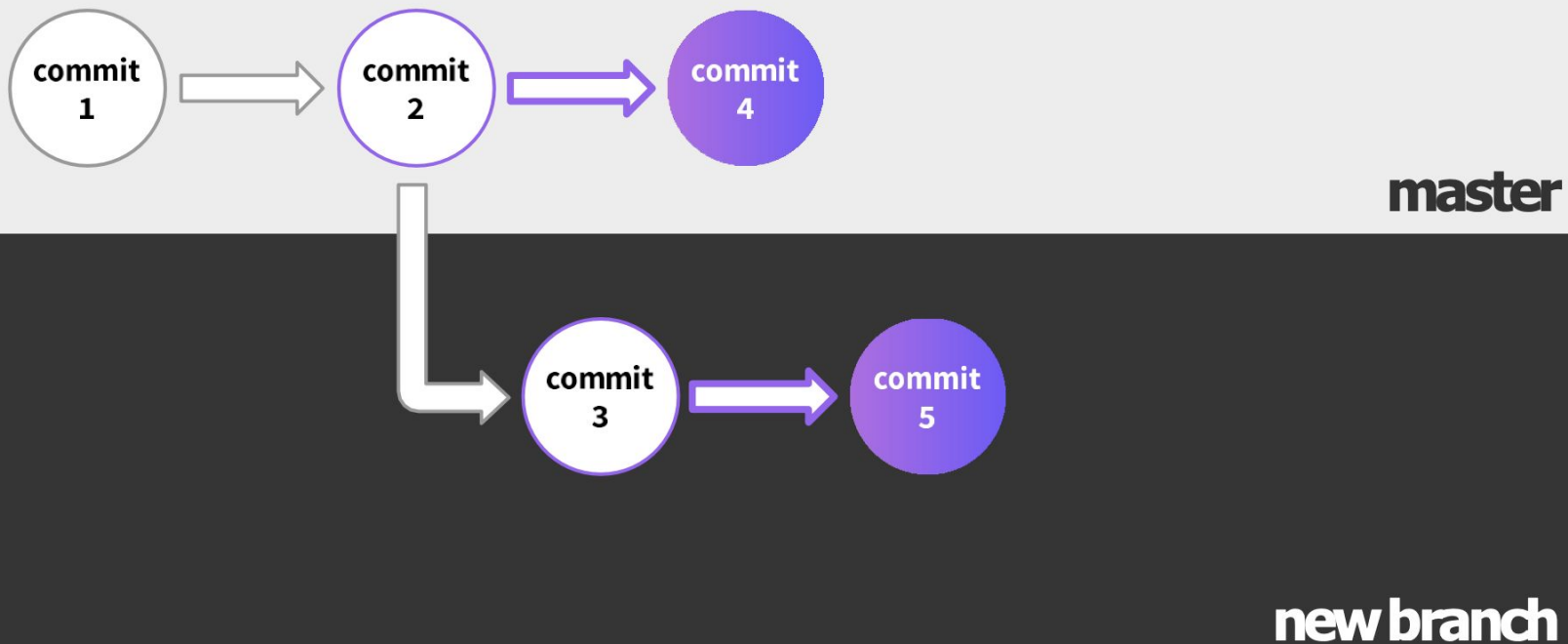
git rebase



master

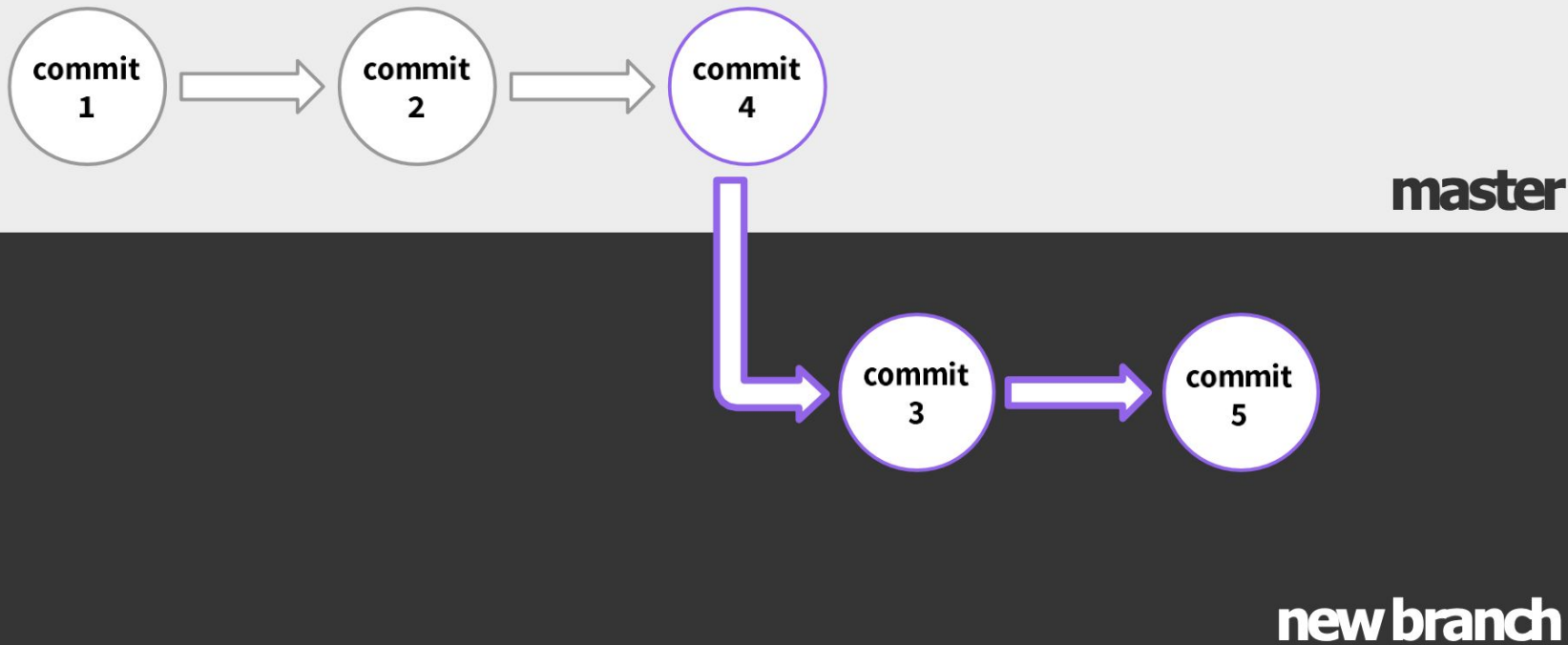
new branch

git rebase



git rebase

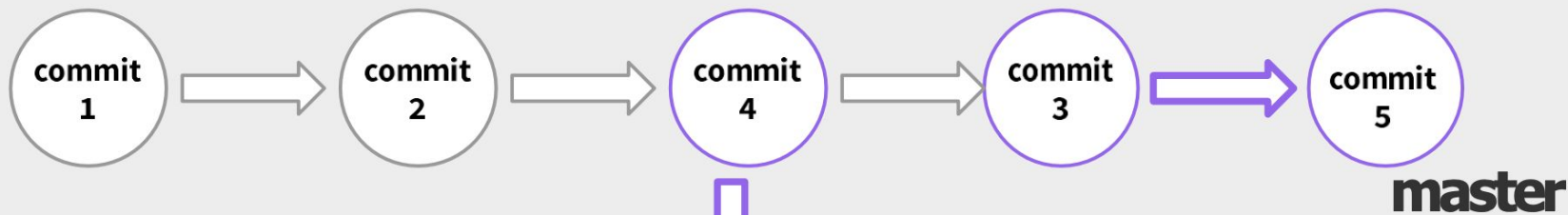
Rebasing new branch ONTO master



git rebase

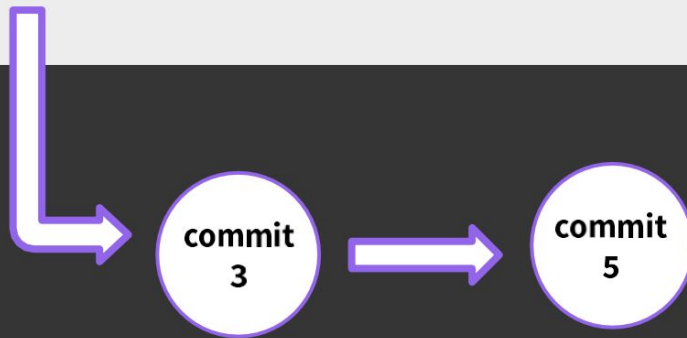
After rebasing we merge master with the new branch (will result in git just "fast forwarding" the master branch for us).

`git checkout master && git merge "new branch"`



Note: We tend to not merge branches through terminal, instead use Pull request. When merging the pull request, we are given an option to

1. Create a merge commit
2. Squash and merge
3. Rebase and merge (What we just saw!)



If you are interested in interactive rebasing, let me know!

new branch

what if I have to undo a commit?

git is designed to be immutable, so **undoing things can get dicey**. You have to be careful and use the right tools!

I would be very cautious playing around with git history!



reverting

undoing your changes



resetting

changes the state of head



restore

restore your changes



stash

store your changes for later

Git Restore



We can restore a file to its state at the previous most recent commit using the git restor

```
> git restore <filename>
```

Think of this command as kind of an ultimate Ctrl + Z which undoes all the changes and restores files to their previous state at the previous commit.

Git Restore actually restores the file back to the head, which typically we have pointing at the most recent commit in the branch.

We can technically point this head wherever we want.

```
>git restore --source HEAD~N file.txt
```

N is the number of commits prior from the head to go back to.

Git Restore also allows us to unstaged files that we had already added to the staging area using get add.

```
>git restore --staged filename
```

Git Reset



Git reset allows us to remove the commits and reset the branch. There are two main types of git reset

```
>git reset #####
```

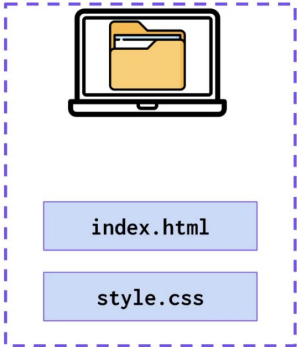
Removes the commits in front of specific hash called, files unchanged

```
>git reset #####
```

Removes the commit and change in the files



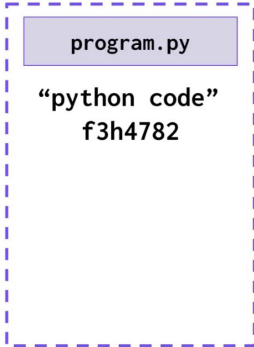
Working Directory



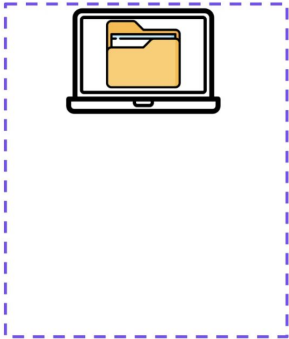
Staging Area



Repository



Working Directory



Staging Area

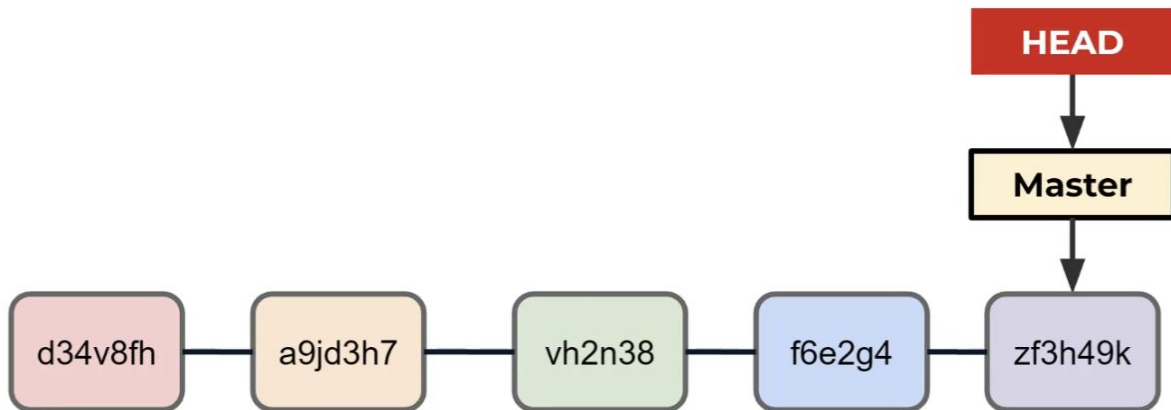


Repository



Git Revert

Git revert will create a new commit that undoes the work from previous commit but keep those commits in the branch.





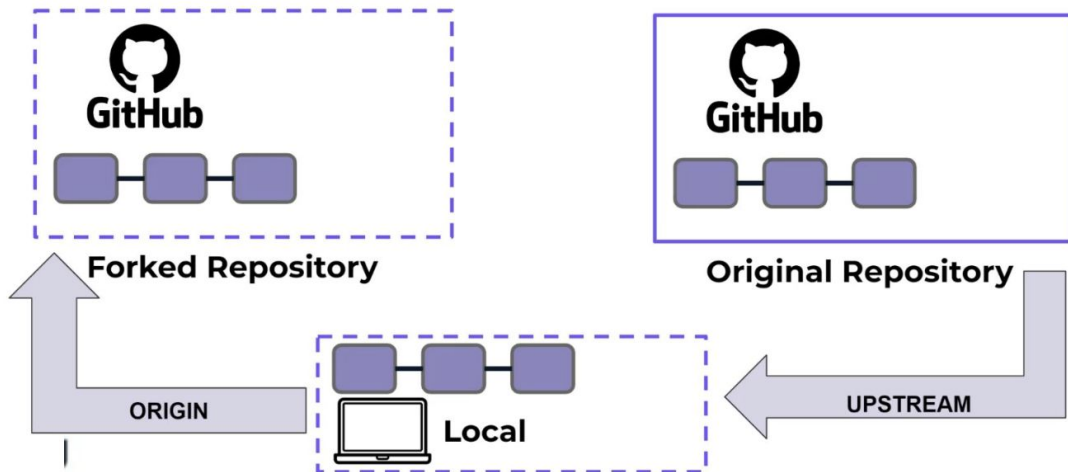
Github Pull Request

- Pull request is a service feature of GitHub, and it's actually not native to git (competing platforms also provide pull request feature)
- Allows for organized review of new branches into a branch

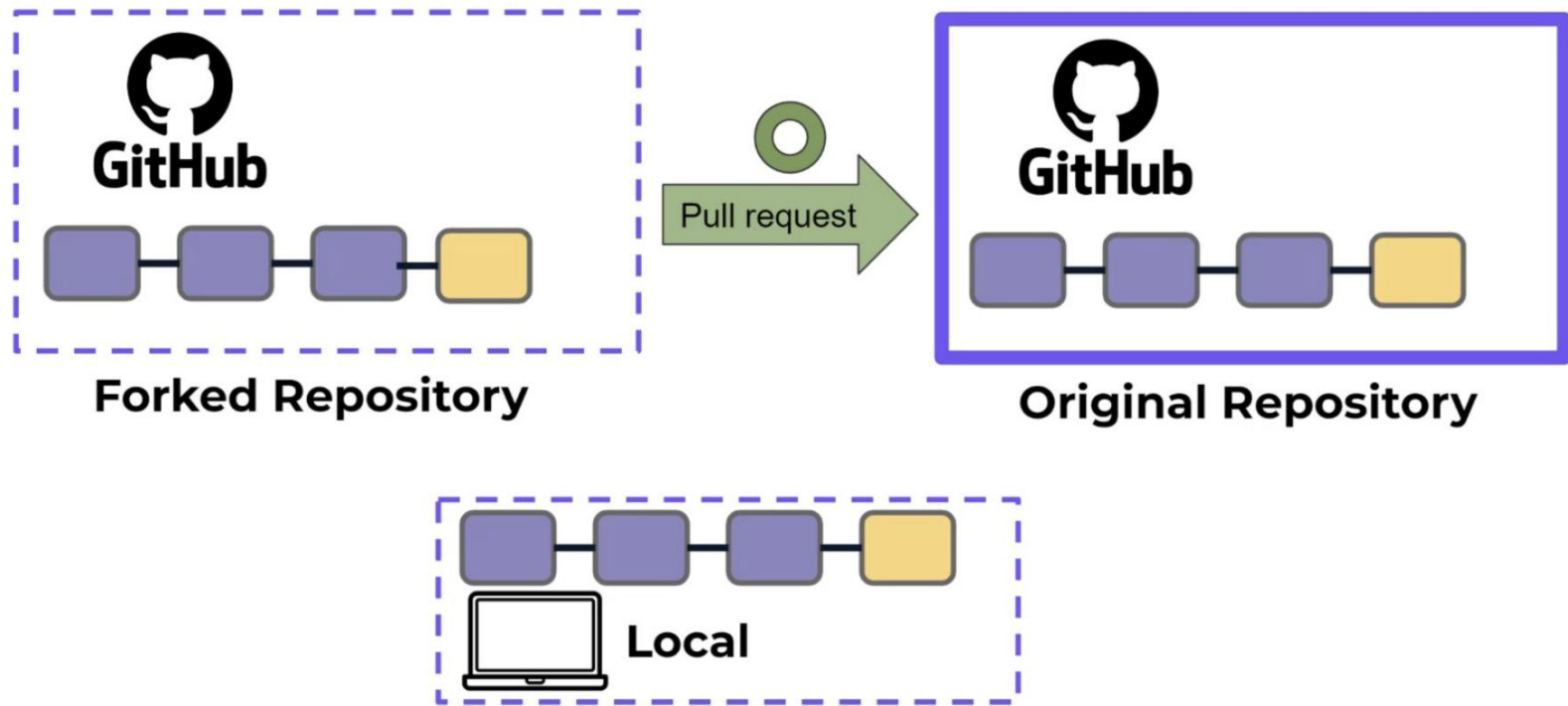
Forking

So now that we understand pull requests, let's explore the workflow of using a fork, which is a GitHub feature, not a git feature, creating a copy of a repository on GitHub to your own GitHub account.

Then you can clone that version locally, make changes and create pull requests to merge those changes.



Forking Contd.





Project advise

- **Use git flow!** It's industry practice for a reason
- When committing, use Semantic Commit Messages ([Link](#))
- use pull request templates ([Link](#))
 - [Example](#)
- have one or two people thoroughly review each pull request
- Create a pull request for each feature, this will keep it very easy to track your progress (make sure to select squash and merge!)
- 1000 commits is not always better than 20 commits ([Example](#))
 - Try to make each commit introduce a new feature / addition / bug fix
 - Each commit should have a working copy of your application



Practise

<https://learngitbranching.js.org/>

Complete all tasks in Introduction Sequences (1-4)



Thank You